

Android e Fuchsia

Dois sistemas da Google, duas filosofias

Grupo 8

Felipe Lazzarini Cunha

Lucas Vieira Resende

Nathan Zoffoli

UFJF — DCC062 Sistemas Operacionais — 2026.1

Julho de 2026

Android: kernel Linux monolítico • **Fuchsia:** microkernel Zircon

1. Apresentação do tema
2. Propósito, evolução e virtualização
3. Tipos de aplicação e uso do kernel
4. Requisitos das aplicações
5. Como os requisitos são atendidos
6. Sistema de interrupções
7. Suporte a threads
8. Exclusão mútua, sincronização e IPC
9. Processos e escalonadores
10. Memória e memória virtual
11. I/O e sistemas de arquivos
12. Características distintivas

1. Apresentação do tema

Dois sistemas operacionais da **Google** com filosofias opostas.

Android

- SO móvel mais usado do mundo
- *Kernel* Linux modificado (AOSP)
- Aberto via AOSP
- Estreia em 2008 (HTC Dream)

Fuchsia

- Construído “do zero”
- Microkernel próprio: **Zircon**
- Segurança por *capacidades*
- Público em 2016; Nest Hub em 2021

Monolítico e maduro × micronúcleo e modular.

2. Propósito de cada sistema

Android

- Plataforma completa para dispositivos móveis
- *Framework* + *runtime* + serviços
- Roda sobre *kernel* Linux adaptado
- Foco: ecossistema amplo de apps

Fuchsia

- SO de propósito geral, experimental
- Modular, atualizável e seguro na base
- Tudo é *componente* com capacidades
- Foco: isolamento e composição

2. Evolução e virtualização

Android

- Dalvik → **ART** (*bytecode* DEX)
- Modularização: Treble e Mainline
- **AVF/pKVM**: VMs protegidas (Microdroid)
- Isola memória mesmo se o host cair

Fuchsia

- Magenta → **Zircon** (2017)
- Componentes, pacotes e FIDL
- **Starnix**: compatibilidade com Linux
- Isolamento por capacidades, não VM

3. Tipos de aplicação e uso do kernel

Android

- Apps *interativos* de terceiros
- Kernel Linux usado em:
 - *smartphones* e *tablets*
 - Android TV, Wear OS
 - Android Automotive (carros)
- Mesmo kernel, muitas classes de aparelho

Fuchsia

- Dispositivos conectados / IoT
- Zircon usado em:
 - *smart displays* (Nest Hub)
 - produtos embarcados da Google
- Presença comercial mais restrita
- Um só núcleo, foco em segurança

Android: apps móveis interativos em muitas formas de dispositivo; Fuchsia: dispositivos conectados, com o Zircon como microkernel comum.

4. Requisitos das aplicações

Android

- Baixo **tempo de resposta** (16,7 ms/quadro)
- **Eficiência energética** (bateria)
- Operar sob **pouca memória**
- Isolamento por aplicativo
- Portabilidade entre hardwares

Fuchsia

- **Segurança** por capacidades
- **Atualizabilidade** sob demanda
- Modularidade e composição
- Escalabilidade entre dispositivos
- Compatibilidade com Linux

Android prioriza o *usuário final*; Fuchsia, a *estrutura* do sistema.

5. Como o Android atende os requisitos

Android

- **Responsividade:** Looper/Handler, InputReader/Dispatcher, Choreographer + VSync; alerta *ANR*
- **Energia:** Doze, SystemSuspend, *wakelocks*, mitigação térmica
- **Memória:** kswapd + lmkd encerram processos menos importantes
- **Segurança:** *sandbox* por UID, SELinux, *Verified Boot*, Binder
- **Portabilidade:** HALs e GKI separam plataforma de *drivers*

5. Como o Fuchsia atende os requisitos

Fuchsia

- **Segurança:** processos sem autoridade; recursos via *handles* (menor privilégio)
- **Atualizabilidade:** software em pacotes independentes, sob demanda
- **Modularidade:** componentes com *manifesto* e capacidades declaradas
- **Escalabilidade:** *build* configurado por produto
- **Compatibilidade:** Starnix implementa a interface de *syscalls* do Linux

6. Sistema de interrupções

Android

- IRQ → **GIC** → CPU
- Dividido em duas metades:
 - *Top Half*: rápido, reconhece a IRQ
 - *Bottom Half*: SoftIRQ, Tasklet, Workqueue
- *Wakeup IRQs* + *wakelocks* p/ acordar

Fuchsia

- *Drivers* no espaço de usuário
- Zircon só mascara e sinaliza o evento
- *Interrupt Object* + *Handle*
- Ciclo: `zx_interrupt_wait` / `ack`
- Falha no *driver* não derruba o núcleo

Linux: falha de *driver* = *kernel panic*. Zircon: só reinicia o componente.

7. Suporte a threads

Android

- *Threads* são do *kernel* Linux (modelo **1:1**)
- Base em *pthread*s; Thread de Java/Kotlin
- *UI thread* + *worker threads*
- `HandlerThread` e co-rotinas (Kotlin)
- Escalonamento fica a cargo do Linux

Fuchsia

- *Thread* é um **objeto do Zircon**
- Um processo contém uma ou mais *threads*
- Escalonadas diretamente pelo micronúcleo
- *pthread*s/C11 sobre `zx_thread`
- *Futexes* para sincronização interna

Ambos usam *threads* de núcleo (1:1); o Android as herda do Linux, o Fuchsia as expõe como objetos do Zircon.

8. Exclusão mútua, sincronização e IPC

Android

- Isolamento: 1 processo/UID por app
- Monitores Java: `wait/notify`
- *UI thread*: `Looper` + `MessageQueue` + `Handler`
- **Binder** IPC + `Service Manager` + `AIDL`

Fuchsia

- Recursos via *handles* do Zircon
- *Channels*: mensagens e *handles*
- *Sockets*: fluxo de *bytes*
- *Futexes* locais ao processo

Android: IPC centralizado no Binder. Fuchsia: canais + capacidades.

9. Processos e escalonadores

Android

- Processos criados pelo **Zygote**
- `fork()` + *copy-on-write*
- Escalonamento é do *kernel* Linux
- Prioridade: primeiro plano → vazio

Fuchsia

- Kernel escalona *threads*, não apps
- Apps = *componentes* via *runners*
- ELF Runner, Web Runner, Starnix
- Ciclo de vida no *Component Manager*

Android: Zygote “aquecido” acelera a abertura de apps compartilhando páginas.

10. Memória e memória virtual

Android

- Paginação sob demanda + ASLR
- **Sem swap em disco** → **ZRAM** (comprimida)
- Métrica **PSS** mede custo real de RAM
- `1mkd` + *OOM score* liberam memória

Fuchsia

- **VMO**: contêiner de páginas físicas
- **VMAR**: árvore do espaço de endereços
- Acesso só com *handle* válido
- Falha de *driver* contida na sua VMAR

11. I/O e sistemas de arquivos

Android

- I/O pela **VFS** do Linux
- **F2FS** e ext4 (flash interno)
- EROFS (partição de sistema, só leitura)
- FAT/exFAT (cartões SD) via FUSE
- *Scoped storage* isola arquivos por app

Fuchsia

- Sistemas de arquivos em *espaço de usuário*
- Acesso via *canais* / FIDL
- **Fxfs**: sistema de arquivos padrão atual
- **Blobfs**: pacotes por *hash* de conteúdo
- FVM gerencia os volumes

Android reaproveita a VFS do Linux (F2FS/ext4/EROFS); no Fuchsia cada FS é um componente isolado (Fxfs, Blobfs).

12. Características distintivas

Android

- **Zygote**: processo “aquecido” com ART
- *fork + copy-on-write* → boot rápido de apps
- **Treble** isola SO do código do fabricante
- **APEX/Mainline**: atualiza módulos pela loja

Fuchsia

- Boot cresce como **árvore de componentes**
- Cada app traz seu próprio *runner*
- **Software efêmero**: pacotes por URL
- Sistema *evergreen*, sem reinstalar

Uma mesma empresa, dois caminhos.

Android

- Monolítico (Linux), maduro e onipresente
- Otimiza a *experiência do usuário*
- Evolui adaptando um núcleo consolidado

Fuchsia

- Micronúcleo (Zircon), moderno e modular
- Otimiza *segurança e atualização*
- Repensa o SO a partir do zero

Desempenho e simplicidade × isolamento e atualizabilidade.

Obrigado!

Grupo 8 — DCC062 Sistemas Operacionais — UFJF — 2026.1